ButterflyFX™ Starfighter Weapon VFX Rendering Guide

Definitive Technical Implementation Reference for WebGL/Three.js Weapon Visual Effects

Author: ButterflyFX Engineering | kensgames.com
Version: 2.0 | April 2026 | Platform: WebGL 2.0 / Three.js r160+
Audience: Developers & AI Coding Assistants (Copilot, etc.)

> ⚠️ **Important — For AI Coding Assistants**
>
> This document is the **single source of truth** for all Starfighter weapon VFX. If you are an AI assistant generating code for weapon effects in this game, follow the patterns in this document exactly. Do not substitute cylinders for billboard quads, do not use point sprites for torpedo trails, and do not skip the multi-layer composite approach for proton torpedoes. Each section contains copy-paste-ready code.

1. Introduction & Architecture Overview

1.1 Purpose

This document is the definitive rendering reference for all four weapon visual effects in the ButterflyFX™ Starfighter game. It provides complete, unambiguous shader

code, Three.js integration classes, and architectural patterns that developers and AI coding assistants can use directly without interpretation or guesswork.

1.2 Target Platform

Parameter	Value
Graphics API	WebGL 2.0
Framework	Three.js r160 or later
Shader Language	GLSL ES 3.00 (via Three.js raw shaders) or GLSL ES 1.00 (via ShaderMaterial)
Target Frame Rate	60 fps (16.67ms frame budget)
Minimum Browser	Chrome 90+, Firefox 90+, Edge 90+, Safari 15.4+

1.3 Performance Budget

Resource	Budget	Notes
Simultaneous Projectiles	200 max	All weapon types combined
Active Particle Emitters	50 max	Includes trails, impacts, muzzle flashes
VFX Draw Calls	10 max	Use instancing and batching aggressively
VFX Texture Memory	16 MB max	Single 2048×2048 atlas + noise textures
VFX Vertex Count	50,000 max	All active VFX geometry combined

1.4 Rendering Pipeline Integration

Weapon VFX integrate into the main render loop at a specific point in the frame sequence:

1. **Scene Update:** Gameplay logic updates positions, fires weapons, detects hits.
2. **VFX Update:** `WeaponVFXManager.update(deltaTime)` advances all active effects — moves projectiles, updates particle systems, ages and recycles expired effects.
3. **Main Scene Render:** Opaque geometry renders first (ships, asteroids, stations).
4. **VFX Render:** All weapon VFX render with `depthWrite: false` and `AdditiveBlending`, sorted back-to-front by distance from camera. Use `renderOrder` to control layering within VFX.
5. **Post-Processing:** EMP distortion pass runs here via `EffectComposer`. Bloom pass (optional) enhances all additive VFX.
6. **UI Overlay:** HUD, targeting reticles, ammo counts render last.

1.5 Coordinate System

Property	Convention
Handedness	Right-handed
Up Axis	Z-up (note: Three.js defaults to Y-up; apply rotation at root level)
Units	Meters (1 unit = 1 meter)
Forward	+Y axis (ship nose direction)
Right	+X axis

1.6 “Store Geometry Not Bytes” Philosophy

Where possible, weapon VFX state is encoded on manifold surfaces rather than as disconnected data arrays. The torpedo trail, for example, stores its history as a continuous triangle strip surface (a 2-manifold ribbon) rather than an array of disconnected point positions. This approach has several benefits:

- GPU-native: geometry can be uploaded and rendered without CPU-side iteration per frame.
- Interpolation-friendly: surface parameterization (UV coordinates) naturally provides smooth gradients for alpha fadeout, color transitions, and width tapering.
- Memory-coherent: vertices in a triangle strip are accessed sequentially, maximizing cache hits.
- Topologically robust: a manifold surface cannot self-intersect or produce rendering artifacts from disconnected elements.

Figure 1.1 — Reference examples of space weapon visual effects: laser beams, plasma projectiles, and energy blasts. ButterflyFX™ Starfighter implements four distinct weapon types, each with unique rendering approaches.

2. Laser Beam Rendering

> ⚠️ **Warning — AI Assistants Read This First**
>
> This is the PRIMARY weapon and the one AI coding assistants struggle with most. The laser beam is **NOT** a `THREE.Line`, **NOT** a cylinder mesh, and **NOT** a `THREE.Sprite`. It is a **billboard quad strip** — two triangles forming a flat quad that always faces the camera. This is the only correct approach for a visually

convincing energy beam.

2.1 Geometry Approach

The laser beam is a single camera-facing quad (two triangles) that stretches from the ship's weapon hardpoint to the maximum range or the point of impact. Key specifications:

Parameter	Value
Geometry Type	Billboard quad (2 triangles, 4 vertices)
Width at Origin	0.3 m
Width at Endpoint	0.15 m (slight taper)
Max Range	500 m (gameplay-defined)
Orientation	Always faces camera (billboard)

The billboard orientation is computed by taking the **cross product** of the beam direction vector and the camera-to-beam-midpoint vector. This yields the quad's local "right" axis, which is used to offset vertices perpendicular to the beam and perpendicular to the view direction.

2.2 GLSL Vertex Shader

laserBeam.vert

```
// Laser Beam Vertex Shader // Billboard quad that stretches from beamStart
to beamEnd // and always faces the camera.  uniform vec3 beamStart; uniform
vec3 beamEnd; uniform float beamWidth; uniform float time; uniform vec3
cameraPosition;  attribute vec3 position; // x: -1 or +1 (side), y: 0 or 1
(along beam), z: unused attribute vec2 uv;  varying vec2 vUv; varying float
vAlpha;  void main() {     // Beam direction and length     vec3 beamDir =
beamEnd - beamStart;     float beamLength = length(beamDir);     vec3
beamDirNorm = beamDir / beamLength;      // Position along the beam (0 =
start, 1 = end)     float t = position.y;      // Point on the beam axis
vec3 pointOnBeam = beamStart + beamDir * t;      // Camera-to-beam vector for
billboarding     vec3 camToBeam = normalize(pointOnBeam - cameraPosition);
// Billboard right axis: perpendicular to both beam and view direction
vec3 right = normalize(cross(beamDirNorm, camToBeam));      // Taper: width
decreases from origin to endpoint     float width = beamWidth * mix(1.0, 0.5,
t);      // Side offset (-1 or +1 from position.x attribute)     vec3 offset
= right * position.x * width * 0.5;      // Sine-wave wobble for energy feel
float wobble = sin(t * 12.0 + time * 8.0) * 0.02 * (1.0 - t);     offset +=
```

```
right * wobble;      // Final world position     vec3 worldPos = pointOnBeam
+ offset;      // Alpha: full at start, slight fade at very end     vAlpha =
1.0 - t * 0.1;      vUv = uv;      gl_Position = projectionMatrix *
viewMatrix * vec4(worldPos, 1.0); }
```

2.3 GLSL Fragment Shader

laserBeam.frag

```
// Laser Beam Fragment Shader // Creates bright white core with colored glow
halo // Scrolling energy pulse, soft edges, additive blending output  uniform
float time; uniform vec3 beamColor;     // e.g., vec3(1.0, 0.2, 0.2) for red
uniform sampler2D noiseTexture;  varying vec2 vUv; varying float vAlpha;
void main() {     // Distance from center axis (UV.x: 0.0 = left edge, 1.0 =
right edge)     float distFromCenter = abs(vUv.x - 0.5) * 2.0;      // Bright
white-hot core (narrow center band)     float core = 1.0 - smoothstep(0.0,
0.15, distFromCenter);      // Colored glow halo (wider, softer falloff)
float glow = 1.0 - smoothstep(0.0, 0.8, distFromCenter);     glow = pow(glow,
2.0);      // Scrolling noise texture for energy pulse effect     vec2
noiseUV = vec2(vUv.y * 3.0 - time * 2.0, vUv.x);     float noise =
texture2D(noiseTexture, noiseUV).r;     float energyPulse = 0.8 + 0.2 *
noise;      // Pulsing intensity     float pulse = 0.9 + 0.1 * sin(time *
15.0);      // Combine: white core + colored glow     vec3 coreColor =
vec3(1.0, 1.0, 1.0) * core;     vec3 glowColor = beamColor * glow;     vec3
finalColor = (coreColor + glowColor) * energyPulse * pulse;      // Edge
fadeout: soft transparency at quad edges     float edgeFade = 1.0 -
smoothstep(0.6, 1.0, distFromCenter);      // Final alpha (premultiplied for
additive blending)     float alpha = (core + glow * 0.6) * edgeFade * vAlpha;
// Output: premultiplied alpha for additive blending     gl_FragColor =
vec4(finalColor * alpha, alpha); }
```

2.4 Three.js Integration Code

LaserBeam.js

```
import * as THREE from 'three'; import laserVertexShader from
'./shaders/laserBeam.vert'; import laserFragmentShader from
'./shaders/laserBeam.frag';  /**  * LaserBeam - Instantaneous hitscan beam
weapon VFX.  *  * Usage:  *   const beam = new LaserBeam(scene,
noiseTexture);  *   beam.fire(shipHardpoint, aimDirection, 500);  *   // In
render loop:  *   beam.update(deltaTime);  *   // On hit detection:  *
beam.hit(hitPoint, hitNormal);  */ export class LaserBeam
{     constructor(scene, noiseTexture, color = new THREE.Vector3(1, 0.2,
0.2)) {         this.scene = scene;         this.active = false;
this.age = 0;         this.holdDuration = 0.1;   // seconds beam stays at
full brightness         this.fadeDuration = 0.15;  // seconds to fade out
after hold         this.state = 'idle'; // idle | hold | fade | done
// -- Billboard Quad Geometry --         // 4 vertices: 2 at beam start
(left/right), 2 at beam end (left/right)         const positions = new
Float32Array([             -1, 0, 0,  // bottom-left  (start, left side)
1, 0, 0,  // bottom-right (start, right side)             -1, 1, 0,  // top-
left     (end, left side)              1, 1, 0   // top-right    (end, right
side)         ]);         const uvs = new Float32Array([             0, 0,
1, 0,             0, 1,             1, 1         ]);         const indices =
new Uint16Array([0, 1, 2, 2, 1, 3]);         this.geometry = new
THREE.BufferGeometry();         this.geometry.setAttribute('position',
new THREE.BufferAttribute(positions, 3));
```

```
this.geometry.setAttribute('uv',             new THREE.BufferAttribute(uvs,
2));         this.geometry.setIndex(new THREE.BufferAttribute(indices, 1));
// -- Shader Material --         this.material = new THREE.ShaderMaterial({
vertexShader: laserVertexShader,             fragmentShader:
laserFragmentShader,             uniforms: {                 beamStart:
{ value: new THREE.Vector3() },                 beamEnd:      { value: new
THREE.Vector3() },                 beamWidth:    { value: 0.3 },
beamColor:    { value: color },                 time:         { value: 0 },
noiseTexture: { value: noiseTexture },                 cameraPosition:
{ value: new THREE.Vector3() }             },             transparent: true,
blending: THREE.AdditiveBlending,             depthWrite: false,
depthTest: true,             side: THREE.DoubleSide         });
this.mesh = new THREE.Mesh(this.geometry, this.material);
this.mesh.frustumCulled = false;         this.mesh.renderOrder = 100;
this.mesh.visible = false;          // -- Muzzle Flash (sprite at origin) --
this.muzzleFlash = this._createMuzzleFlash(color);
this.muzzleFlash.visible = false;          // -- Impact Effect (sprite at hit
point) --         this.impactSprite = this._createImpactSprite(color);
this.impactSprite.visible = false;          scene.add(this.mesh);
scene.add(this.muzzleFlash);         scene.add(this.impactSprite);     }
/**      * Fire the beam from origin along direction.      * Beam appears at
full length immediately (hitscan).      */     fire(origin, direction,
maxRange = 500) {         const dir = direction.clone().normalize();
const end = origin.clone().add(dir.multiplyScalar(maxRange));
this.material.uniforms.beamStart.value.copy(origin);
this.material.uniforms.beamEnd.value.copy(end);
this.material.uniforms.beamWidth.value = 0.3;          this.mesh.visible =
true;         this.active = true;         this.age = 0;         this.state =
'hold';          // Muzzle flash
this.muzzleFlash.position.copy(origin);         this.muzzleFlash.visible =
true;         this.muzzleFlashAge = 0;     }      /**      * Register a hit
at the given point. Shortens beam and spawns impact.      */     hit(point,
normal) {         this.material.uniforms.beamEnd.value.copy(point);
this.impactSprite.position.copy(point);         this.impactSprite.visible =
true;         this.impactAge = 0;     }      /**      * Call every frame with
deltaTime in seconds.      */     update(deltaTime) {         if (!
this.active) return;          this.age += deltaTime;
this.material.uniforms.time.value += deltaTime;          // Update camera
position for billboard calculation         // (Set this from your camera
reference)         //
this.material.uniforms.cameraPosition.value.copy(camera.position);          /
/ State machine         switch (this.state) {             case 'hold':
if (this.age >= this.holdDuration) {                     this.state = 'fade';
this.fadeStart = this.age;                 }                 break;
case 'fade':                 const fadeProgress =
(this.age - this.fadeStart) / this.fadeDuration;
this.material.uniforms.beamWidth.value =                     0.3 * (1.0 -
fadeProgress);                 if (fadeProgress >= 1.0) {
this.state = 'done';                     this.mesh.visible = false;
this.active = false;                 }                 break;         }
// Muzzle flash: 0.05s duration         if (this.muzzleFlash.visible) {
this.muzzleFlashAge += deltaTime;             const muzzleScale = 1.0 +
this.muzzleFlashAge * 20;             this.muzzleFlash.scale.set(muzzleScale,
muzzleScale, 1);             if (this.muzzleFlashAge > 0.05) {
this.muzzleFlash.visible = false;             }         }          // Impact
effect: 0.3s expanding ring         if (this.impactSprite.visible) {
this.impactAge += deltaTime;             const impactScale = 1.0 +
this.impactAge * 8;             this.impactSprite.scale.set(impactScale,
impactScale, 1);             if (this.impactAge > 0.3)
{                 this.impactSprite.visible = false;             }         }
```

```
}      /**      * Clean up all GPU resources.      */     dispose() {
this.scene.remove(this.mesh);         this.scene.remove(this.muzzleFlash);
this.scene.remove(this.impactSprite);         this.geometry.dispose();
this.material.dispose();     }      _createMuzzleFlash(color) {         const
spriteMat = new THREE.SpriteMaterial({             color: 0xFFFFAA,
blending: THREE.AdditiveBlending,             depthWrite: false,
transparent: true         });         const sprite = new
THREE.Sprite(spriteMat);         sprite.scale.set(0.6, 0.6, 1); // 2x beam
width         sprite.renderOrder = 101;         return sprite;     }
_createImpactSprite(color) {         const spriteMat = new
THREE.SpriteMaterial({             color: new THREE.Color(color.x, color.y,
color.z),             blending: THREE.AdditiveBlending,
depthWrite: false,             transparent: true         });         const
sprite = new THREE.Sprite(spriteMat);         sprite.scale.set(0.5, 0.5, 1);
sprite.renderOrder = 102;         return sprite;     } }
```

2.5 Visual Reference

Property	Specification
Core Color	Bright white (#FFFFFF)
Glow Envelope	Weapon-colored (red, green, or blue)
Behavior	Instantaneous hitscan — appears full length immediately
Hold Duration	0.1 seconds at full brightness
Fade Duration	0.15 seconds (width shrinks to zero)
Muzzle Flash	0.05s duration, 2× beam width, white-yellow (#FFFFAA)
Impact Flash	Billboard sprite, 0.1s duration, beam-colored
Impact Ring	Expanding ring, 0.3s duration
Impact Sparks	8–12 particles, hemispherical spread along hit normal

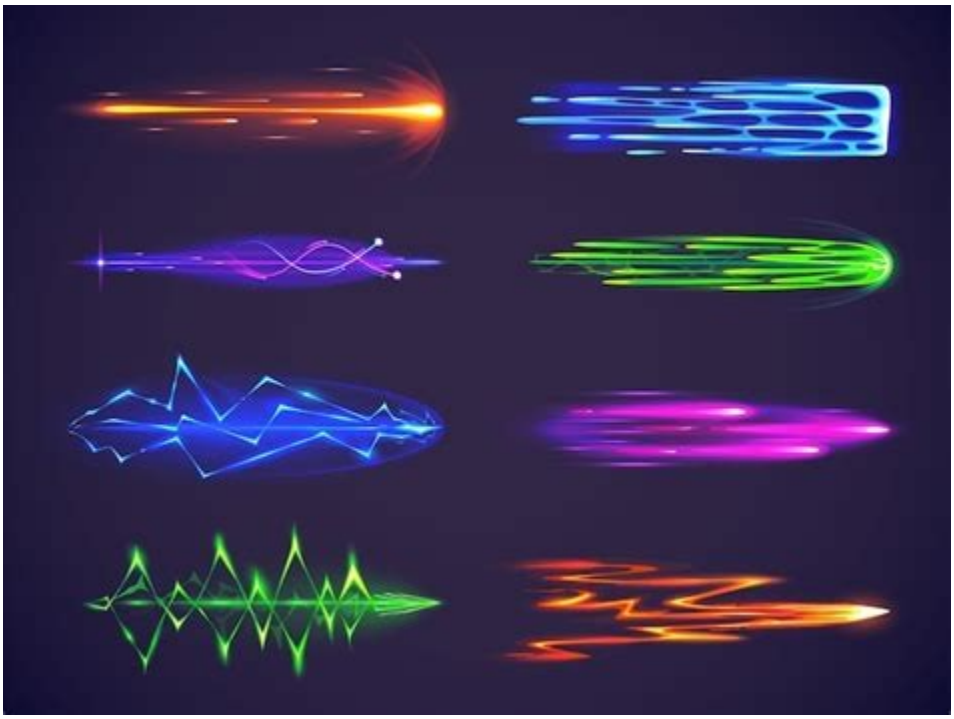

Figure 2.1 — Laser beam visual effect anatomy: the billboard quad creates a camera-facing rectangle with a white-hot core and colored glow envelope, plus scrolling energy noise along its length.

3. Scatter Shot Rendering

The Scatter Shot is a shotgun-style spread of small, fast-moving projectiles. It trades single-target accuracy for wide area coverage at close range.

3.1 Geometry Approach

Parameter	Value
Geometry Type	Billboard sprites via `InstancedMesh`
Pellet Diameter	0.1 m
Pellets Per Shot	8–12 (randomized per fire event)
Spread Cone	15° half-angle
Speed	200 m/s base, ±10% per pellet
Max Range	100 m
Trail	3–4 previous positions per pellet, rendered as smaller fading sprites

All pellets in a single shot are rendered with one `InstancedMesh` draw call. Each pellet's position, scale, and color are updated per frame via the instance matrix and instance color attributes.

3.2 GLSL Shaders

scatterPellet.vert

```
// Scatter Shot Pellet Vertex Shader // Standard billboard with size
attenuation over distance  uniform float time;  attribute vec3 position;
attribute vec2 uv;  varying vec2 vUv; varying float vFlicker;  void main() {
vUv = uv;      // Flicker: random-ish per-instance variation using instance
position     vec4 mvPosition = modelViewMatrix * instanceMatrix * vec4(0.0,
0.0, 0.0, 1.0);     float dist = -mvPosition.z;      // Size attenuation:
smaller when farther away     float sizeAtten = 300.0 / dist;     sizeAtten =
clamp(sizeAtten, 0.5, 3.0);      // Billboard: offset in screen space
mvPosition.xy += position.xy * 0.05 * sizeAtten;      // Flicker noise
(simple hash from distance + time)     vFlicker = 0.85 + 0.15 * sin(dist *
73.17 + time * 25.0);      gl_Position = projectionMatrix * mvPosition; }
```

scatterPellet.frag

```
// Scatter Shot Pellet Fragment Shader // Radial gradient, orange-yellow,
flickering  uniform float time; uniform vec3 pelletColor; // vec3(1.0, 0.7,
0.2) orange-yellow  varying vec2 vUv; varying float vFlicker;  void main() {
// Radial distance from center     vec2 centered = vUv - 0.5;     float dist
= length(centered) * 2.0;      // Soft circular gradient     float brightness
= 1.0 - smoothstep(0.0, 1.0, dist);     brightness = pow(brightness, 1.5);
// Bright center     float core = 1.0 - smoothstep(0.0, 0.3, dist);      //
Combine     vec3 color = mix(pelletColor, vec3(1.0, 1.0, 0.9), core);
float alpha = brightness * vFlicker;      gl_FragColor = vec4(color * alpha,
alpha); }
```

3.3 Three.js Integration Code

ScatterShot.js

```
import * as THREE from 'three'; import scatterVertShader from
'./shaders/scatterPellet.vert'; import scatterFragShader from
'./shaders/scatterPellet.frag';  const MAX_PELLETS = 12; const TRAIL_LENGTH =
4; const _tempMatrix = new THREE.Matrix4(); const _tempVec = new
THREE.Vector3();  /**  * ScatterShot - Shotgun-style spread weapon VFX.  *
Uses InstancedMesh for all pellets in one draw call.  *  * Usage:  *   const
scatter = new ScatterShot(scene);  *   scatter.fire(origin, direction, 12);
*   // In render loop:  *   scatter.update(deltaTime);  */ export class
ScatterShot {     constructor(scene) {         this.scene = scene;
this.active = false;         this.pellets = [];          // Billboard quad
geometry (small plane)         const planeGeo = new THREE.PlaneGeometry(1,
1);          // Shader material         this.material = new
THREE.ShaderMaterial({             vertexShader: scatterVertShader,
fragmentShader: scatterFragShader,             uniforms:
{                 time: { value: 0 },                 pelletColor: { value:
new THREE.Vector3(1.0, 0.7, 0.2) }             },             transparent:
true,             blending: THREE.AdditiveBlending,             depthWrite:
false,             side: THREE.DoubleSide         });          //
InstancedMesh: MAX_PELLETS instances for pellets         this.pelletMesh =
new THREE.InstancedMesh(             planeGeo, this.material, MAX_PELLETS
```

```
);         this.pelletMesh.instanceMatrix.setUsage(THREE.DynamicDrawUsage);
this.pelletMesh.frustumCulled = false;         this.pelletMesh.renderOrder =
100;         this.pelletMesh.visible = false;         this.pelletMesh.count =
0;          // Trail InstancedMesh: MAX_PELLETS * TRAIL_LENGTH
this.trailMesh = new THREE.InstancedMesh(             planeGeo,
this.material,             MAX_PELLETS * TRAIL_LENGTH         );
this.trailMesh.instanceMatrix.setUsage(THREE.DynamicDrawUsage);
this.trailMesh.frustumCulled = false;         this.trailMesh.renderOrder =
99;         this.trailMesh.visible = false;         this.trailMesh.count = 0;
scene.add(this.pelletMesh);         scene.add(this.trailMesh);     }      /**
* Fire a scatter shot burst.      * @param {THREE.Vector3} origin - Hardpoint
position      * @param {THREE.Vector3} direction - Aim direction (normalized)
* @param {number} count - Number of pellets (8-12)      */     fire(origin,
direction, count = 10) {         const halfAngle =
THREE.MathUtils.degToRad(15);         this.pellets = [];          for (let i
= 0; i < count; i++) {             // Random direction within cone
const theta = Math.random() * Math.PI * 2;             const phi =
Math.random() * halfAngle;             const spreadDir = direction.clone();
// Create perpendicular axes             const up = Math.abs(direction.y) <
0.9                 ? new THREE.Vector3(0, 1, 0)                 : new
THREE.Vector3(1, 0, 0);             const right = new THREE.Vector3()
.crossVectors(direction, up).normalize();             const orthoUp = new
THREE.Vector3()                 .crossVectors(right, direction).normalize();
// Apply cone spread
spreadDir.add(                 right.clone().multiplyScalar(
Math.sin(phi) * Math.cos(theta))             );             spreadDir.add(
orthoUp.clone().multiplyScalar(                     Math.sin(phi) *
Math.sin(theta))             );             spreadDir.normalize();
// Speed variance: base 200 m/s +/- 10%             const speed = 200 * (0.9
+ Math.random() * 0.2);
this.pellets.push({                 position: origin.clone(),
velocity: spreadDir.multiplyScalar(speed),                 age: 0,
alive: true,                 trail: [] // stores last TRAIL_LENGTH positions
});         }          this.pelletMesh.count = count;
this.pelletMesh.visible = true;         this.trailMesh.visible = true;
this.active = true;     }      /**      * Update all pellets. Call once per
frame.      */     update(deltaTime) {         if (!this.active) return;
this.material.uniforms.time.value += deltaTime;         let allDead = true;
let trailIndex = 0;          for (let i = 0; i < this.pellets.length; i++) {
const p = this.pellets[i];             if (!p.alive)
continue;              // Store trail position before moving
p.trail.unshift(p.position.clone());             if (p.trail.length >
TRAIL_LENGTH) p.trail.pop();              // Move pellet
_tempVec.copy(p.velocity).multiplyScalar(deltaTime);
p.position.add(_tempVec);             p.age += deltaTime;              // Max
range check (100m = ~0.5s at 200 m/s)             if (p.age > 0.5) {
p.alive = false;                 _tempMatrix.makeScale(0, 0, 0);
this.pelletMesh.setMatrixAt(i, _tempMatrix);                 continue;
}              allDead = false;              // Update pellet instance matrix
(position + scale)
_tempMatrix.makeTranslation(                 p.position.x, p.position.y,
p.position.z             );             this.pelletMesh.setMatrixAt(i,
_tempMatrix);              // Update trail instances             for (let t =
0; t < p.trail.length; t++) {                 const trailPos = p.trail[t];
const trailScale = 1.0 - (t + 1) / (TRAIL_LENGTH + 1);
_tempMatrix.makeTranslation(                     trailPos.x, trailPos.y,
trailPos.z                 );                 _tempMatrix.scale(
_tempVec.set(trailScale, trailScale, trailScale)                 );
if (trailIndex < MAX_PELLETS * TRAIL_LENGTH)
{                     this.trailMesh.setMatrixAt(trailIndex, _tempMatrix);
```

```
trailIndex++;                 }             }         }
this.pelletMesh.instanceMatrix.needsUpdate = true;
this.trailMesh.count = trailIndex;
this.trailMesh.instanceMatrix.needsUpdate = true;          if (allDead) {
this.active = false;             this.pelletMesh.visible = false;
this.trailMesh.visible = false;         }     }      /**      * Clean up GPU
resources.      */     dispose()
{         this.scene.remove(this.pelletMesh);
this.scene.remove(this.trailMesh);
this.pelletMesh.geometry.dispose();         this.material.dispose();     } }
```

4. Proton Torpedo Rendering

> ⚠️ **Warning — This Is the HARDEST to Render**
>
> AI coding assistants consistently fail on this weapon. The torpedo is **NOT** a single mesh with a particle trail. It is a **4-layer composite** — core projectile, glow shell, engine trail (ribbon geometry), and particle wake — all managed as children of one `Object3D`. The engine trail **MUST** be a triangle strip ribbon, not a series of point sprites. Read every subsection carefully.

4.1 Geometry Approach — Multi-Layer Composite

The proton torpedo is composed of four distinct rendering layers, each with its own geometry and shader. They are attached as children of a single `THREE.Object3D` so they move together.

Layer	Geometry Type	Size	Blending	Purpose
1. Core Projectile	Elongated SphereGeometry (0.5 × 0.2 × 0.2 m)	Small, solid	Normal	Visible torpedo body
2. Glow Shell	Billboard Sprite (2 m diameter)	4× core diameter	Additive	Volumetric glow envelope

Layer	Geometry Type	Size	Blending	Purpose
3. Engine Trail	Triangle Strip Ribbon (30 segments)	Tapers 0.4 m → 0.0 m	Additive	Continuous energy wake
4. Particle Wake	Points geometry (GPU particles)	0.05 m → 0.2 m	Additive	Scattered exhaust debris

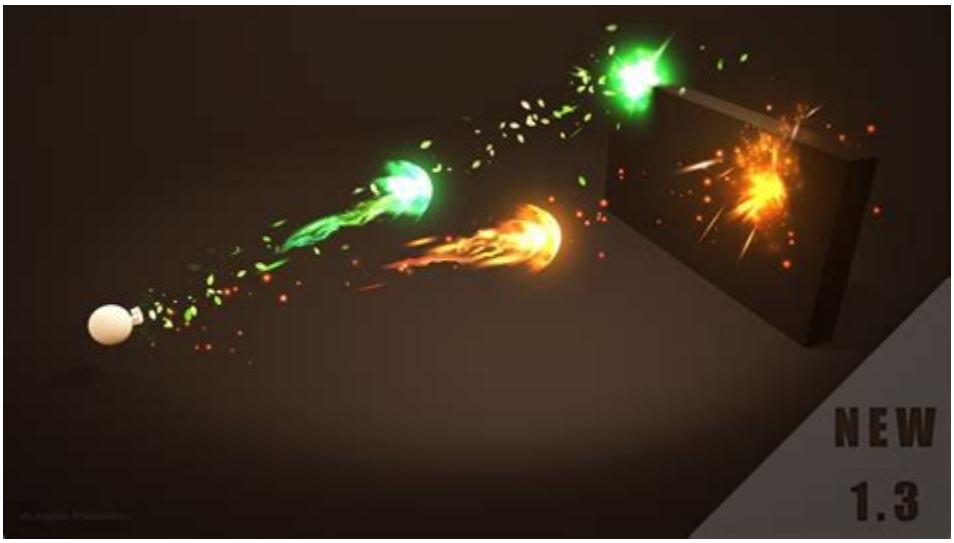

Figure 4.1 — Multi-layer projectile VFX composition: a solid core surrounded by a billboard glow shell, followed by a continuous ribbon trail and scattered particle wake. The ButterflyFX proton torpedo uses this exact layered architecture.

4.2 Core Projectile Shader

torpedoCore.vert

```
// Torpedo Core Vertex Shader // Standard MVP with slow spin around forward axis + Fresnel prep  uniform float time; uniform float spinRate; // radians per second  attribute vec3 position; attribute vec3 normal; attribute vec2 uv;  varying vec3 vNormal; varying vec3 vViewDir; varying vec2 vUv;  void main() {     // Spin around local Z axis (forward axis)     float angle = time * spinRate;     float c = cos(angle);     float s = sin(angle);     vec3 spunPos = vec3(         position.x * c - position.y * s,         position.x * s + position.y * c,         position.z     );     vec3 spunNormal = vec3( normal.x * c - normal.y * s,         normal.x * s + normal.y * c, normal.z     );      vec4 worldPos = modelMatrix * vec4(spunPos, 1.0); vec4 mvPos = viewMatrix * worldPos;      vNormal = normalize(normalMatrix * spunNormal);     vViewDir = normalize(-mvPos.xyz);     vUv = uv; gl_Position = projectionMatrix * mvPos; }
```

torpedoCore.frag

```
// Torpedo Core Fragment Shader // Fresnel rim glow + pulsing emissive, blue-white body  uniform float time; uniform vec3 coreColor;  // vec3(0.53, 0.80, 1.0)  = #88CCFF uniform vec3 rimColor;   // vec3(0.13, 0.27, 1.0)  = #2244FF
```

```
varying vec3 vNormal; varying vec3 vViewDir; varying vec2 vUv;  void main() {
// Fresnel: bright at edges (rim), dimmer at center     float fresnel = 1.0 -
max(dot(vNormal, vViewDir), 0.0);     fresnel = pow(fresnel, 3.0);      //
Pulsing emissive intensity     float pulse = 0.85 + 0.15 * sin(time * 12.0);
// Mix core and rim colors based on Fresnel     vec3 color = mix(coreColor,
rimColor, fresnel);     color *= pulse;      // Emissive output (no lighting
needed, self-illuminated)     float alpha = 1.0;     gl_FragColor =
vec4(color, alpha); }
```

4.3 Glow Shell Shader

torpedoGlow.frag

```
// Torpedo Glow Shell Fragment Shader // Large billboard sprite, radial
gradient with exponential falloff // Pulsing size handled by CPU (scale
uniform), color shift center to edge  uniform float time;  varying vec2 vUv;
void main() {     // Radial distance from center     vec2 centered = vUv -
0.5;     float dist = length(centered) * 2.0;      // Exponential falloff for
volumetric feel     float glow = exp(-dist * dist * 3.0);      // Pulsing
intensity     float pulse = 0.8 + 0.2 * sin(time * 8.0);     glow *= pulse;
// Color: blue-white at center, deep blue at edges     vec3 centerColor =
vec3(0.7, 0.85, 1.0);  // blue-white     vec3 edgeColor = vec3(0.1, 0.2,
0.8);     // deep blue     vec3 color = mix(centerColor, edgeColor, dist);
// Alpha: fade at edge of sprite     float alpha = glow * smoothstep(1.0,
0.6, dist);      gl_FragColor = vec4(color * alpha, alpha); }
```

4.4 Engine Trail — Ribbon Geometry

> ⚠️ **Critical Implementation Detail**
>
> The engine trail is a **TRIANGLE STRIP**, not a series of sprites or points. This is the most common mistake AI assistants make. A triangle strip creates a smooth, continuous ribbon surface that tapers and fades naturally. Using point sprites produces a “dotted line” effect that looks wrong. The geometry is rebuilt each frame from a ring buffer of position history.

RibbonTrail.js — Engine Trail Geometry Builder

```
import * as THREE from 'three';  const MAX_TRAIL_POINTS = 30; const
VERTS_PER_POINT = 2; // left + right const TOTAL_VERTS = MAX_TRAIL_POINTS *
VERTS_PER_POINT;  /**  * RibbonTrail - Triangle strip ribbon that follows a
moving object.  * Stores position history in a ring buffer and rebuilds strip
each frame.  */ export class RibbonTrail {     constructor(scene, camera) {
this.scene = scene;         this.camera = camera;          // Ring buffer for
position history         this.positions = new
Array(MAX_TRAIL_POINTS).fill(null);         this.head = 0;       // write
index         this.count = 0;      // number of valid positions
this.headWidth = 0.4; // meters         this.tailWidth = 0.0; // meters (full
taper)          // Pre-allocate BufferGeometry         this.posAttr = new
```

```
Float32Array(TOTAL_VERTS * 3);         this.uvAttr = new
Float32Array(TOTAL_VERTS * 2);         this.indexAttr = [];          // Build
index array for triangle strip as indexed triangles         for (let i = 0; i
< MAX_TRAIL_POINTS - 1; i++) {             const bl = i * 2;
const br = i * 2 + 1;             const tl = (i + 1) * 2;             const
tr = (i + 1) * 2 + 1;             // Two triangles per segment
this.indexAttr.push(bl, br, tl);             this.indexAttr.push(tl, br, tr);
}          this.geometry = new THREE.BufferGeometry();
this.geometry.setAttribute('position',             new
THREE.BufferAttribute(this.posAttr,
3)                 .setUsage(THREE.DynamicDrawUsage));
this.geometry.setAttribute('uv',             new
THREE.BufferAttribute(this.uvAttr,
2)                 .setUsage(THREE.DynamicDrawUsage));
this.geometry.setIndex(this.indexAttr);          // Shader material
this.material = new THREE.ShaderMaterial({             vertexShader: `
attribute vec3 position;                 attribute vec2 uv;
varying vec2 vUv;                 void main() {                     vUv = uv;
gl_Position = projectionMatrix                         * modelViewMatrix *
vec4(position, 1.0);                 }             `,
fragmentShader: `                 uniform float time;                 uniform
sampler2D noiseTexture;                 varying vec2 vUv;
void main() {                     // UV.y: 0.0 at head, 1.0 at tail
float t = vUv.y;                     // Alpha fadeout toward tail
float alpha = 1.0 - smoothstep(0.0, 1.0, t);                     alpha =
pow(alpha, 1.5);                     // Scrolling energy pattern along trail
vec2 noiseUV = vec2(                         vUv.x * 2.0,
t * 4.0 - time * 3.0                     );                     float noise =
texture2D(noiseTexture, noiseUV).r;                     float energy = 0.7 +
0.3 * noise;                     // Color gradient: blue-cyan at head to
deep blue at tail                     vec3 headColor = vec3(0.4, 0.8,
1.0); // cyan                     vec3 tailColor = vec3(0.1, 0.15, 0.6); //
deep blue                     vec3 color = mix(headColor, tailColor, t);
color *= energy;                     alpha *= energy;
gl_FragColor = vec4(color * alpha, alpha);                 }             `,
uniforms: {                 time: { value: 0 },                 noiseTexture:
{ value: null } // assign noise texture             },
transparent: true,             blending: THREE.AdditiveBlending,
depthWrite: false,             side: THREE.DoubleSide         });
this.mesh = new THREE.Mesh(this.geometry, this.material);
this.mesh.frustumCulled = false;         this.mesh.renderOrder = 98;
scene.add(this.mesh);     }      /**      * Push the current torpedo position
into the ring buffer.      * Call once per frame while torpedo is alive.
*/     pushPosition(pos) {         this.positions[this.head] = pos.clone();
this.head = (this.head + 1) % MAX_TRAIL_POINTS;         if (this.count <
MAX_TRAIL_POINTS) this.count++;     }      /**      * Rebuild the triangle
strip geometry from the ring buffer.      * Must be called after
pushPosition() each frame.      */     rebuild(deltaTime)
{         this.material.uniforms.time.value += deltaTime;          if
(this.count < 2) return;          const camPos = this.camera.position;
for (let i = 0; i < this.count; i++) {             // Read from ring buffer:
head-1 is newest, walk backward             const bufIdx =
((this.head - 1 - i) + MAX_TRAIL_POINTS * 2)                 %
MAX_TRAIL_POINTS;             const pos = this.positions[bufIdx];
if (!pos) continue;             // t: 0 at head (newest), 1 at tail (oldest)
const t = i / (this.count - 1);             // Width taper             const
width = THREE.MathUtils.lerp(                 this.headWidth, this.tailWidth,
t             );             // Billboard per-segment: get direction to next
point             let dir;             if (i < this.count - 1) {
const nextIdx =                     ((this.head - 2 - i) + MAX_TRAIL_POINTS *
```

```
2)                    % MAX_TRAIL_POINTS;                 const nextPos =
this.positions[nextIdx];                 dir = new THREE.Vector3()
.subVectors(pos, nextPos).normalize();             } else
{                 // Last point: reuse previous direction
const prevIdx =                     ((this.head - i) + MAX_TRAIL_POINTS * 2)
% MAX_TRAIL_POINTS;                 const prevPos = this.positions[prevIdx];
dir = new THREE.Vector3()                     .subVectors(prevPos,
pos).normalize();             }              // Camera-facing right vector
const camToPoint = new THREE.Vector3()                 .subVectors(pos,
camPos).normalize();             const right = new THREE.Vector3()
.crossVectors(dir, camToPoint).normalize();              // Left and right
vertices             const leftIdx = i * VERTS_PER_POINT * 3;
const rightIdx = leftIdx + 3;              this.posAttr[leftIdx]     = pos.x
- right.x * width * 0.5;             this.posAttr[leftIdx + 1] = pos.y -
right.y * width * 0.5;             this.posAttr[leftIdx + 2] = pos.z -
right.z * width * 0.5;              this.posAttr[rightIdx]     = pos.x +
right.x * width * 0.5;             this.posAttr[rightIdx + 1] = pos.y +
right.y * width * 0.5;             this.posAttr[rightIdx + 2] = pos.z +
right.z * width * 0.5;              // UVs: x = 0 (left) / 1 (right), y = t
(head to tail)             const uvLeft = i * VERTS_PER_POINT * 2;
this.uvAttr[uvLeft]     = 0.0;             this.uvAttr[uvLeft + 1] = t;
this.uvAttr[uvLeft + 2] = 1.0;             this.uvAttr[uvLeft + 3] = t;
}          // Update draw range         this.geometry.setDrawRange(0,
(this.count - 1) * 6);         this.geometry.attributes.position.needsUpdate
= true;         this.geometry.attributes.uv.needsUpdate = true;     }
/**      * Reset the trail (on torpedo detonation or disposal).      */
reset() {         this.positions.fill(null);         this.head = 0;
this.count = 0;     }      dispose() {         this.scene.remove(this.mesh);
this.geometry.dispose();         this.material.dispose();     } }
```

4.5 Particle Wake

Parameter	Value
Spawn Rate	20–40 particles per second
Initial Velocity	Torpedo velocity × 0.1 + random radial spread
Initial Size	0.05 m
Final Size	0.2 m (grows over lifetime)
Lifetime	0.3–0.5 seconds
Color Start	Blue-white (#AADDFF)
Color End	Deep blue (#1122AA), then transparent
Geometry	`THREE.Points` with custom shader

The particle wake uses the `GPUParticleSystem` class described in Section 8. Particles are spawned at the torpedo's current position each frame, with slight random radial offsets, and fade through the blue color spectrum before disappearing.

4.6 Three.js Integration Code

ProtonTorpedo.js

```
import * as THREE from 'three'; import { RibbonTrail } from './RibbonTrail';
import { GPUParticleSystem } from './GPUParticleSystem';  const _tempVec =
new THREE.Vector3();  /**  * ProtonTorpedo - Multi-layer composite projectile
with guidance.  *  * 4 rendering layers:  *   1. Core projectile (elongated
sphere, Fresnel shader)  *   2. Glow shell (billboard sprite, pulsing)  *
3. Engine trail (ribbon geometry, triangle strip)  *   4. Particle wake (GPU
particles)  *  * Usage:  *   const torpedo = new ProtonTorpedo(scene, camera,
noiseTexture);  *   torpedo.fire(launchPos, launchDir, targetObject);  *   //
In render loop:  *   torpedo.update(deltaTime);  */ export class
ProtonTorpedo {     constructor(scene, camera, noiseTexture)
{         this.scene = scene;         this.camera = camera;
this.active = false;         this.age = 0;         this.speed = 80;
// m/s         this.straightDuration = 0.5;  // seconds before guidance kicks
in         this.target = null;         this.position = new THREE.Vector3();
this.velocity = new THREE.Vector3();         this.state = 'idle'; // idle |
straight | guided | detonating | done          // -- Container Object3D --
this.container = new THREE.Object3D();         this.container.visible =
false;         scene.add(this.container);          // -- Layer 1: Core
Projectile --         const coreGeo = new THREE.SphereGeometry(1, 16, 12);
coreGeo.scale(0.5, 0.2, 0.2); // elongated         this.coreMaterial = new
THREE.ShaderMaterial({             // Use torpedoCore.vert / .frag from
Section 4.2             vertexShader: /* torpedoCore vertex shader string */`
uniform float time;                 uniform float spinRate;
attribute vec3 position;                 attribute vec3 normal;
varying vec3 vNormal;                 varying vec3 vViewDir;
void main() {                     float angle = time * spinRate;
float c = cos(angle); float s = sin(angle);                     vec3 sp =
vec3(position.x*c - position.y*s,
position.x*s + position.y*c,                                    position.z);
vec3 sn = vec3(normal.x*c - normal.y*s,
normal.x*s + normal.y*c,                                    normal.z);
vec4 wp = modelMatrix * vec4(sp, 1.0);                     vec4 mv =
viewMatrix * wp;                     vNormal = normalize(normalMatrix * sn);
vViewDir = normalize(-mv.xyz);                     gl_Position =
projectionMatrix * mv;                 }             `,
fragmentShader: /* torpedoCore fragment shader string */`
uniform float time;                 varying vec3 vNormal;
varying vec3 vViewDir;                 void main()
{                     float fresnel = 1.0 - max(dot(vNormal, vViewDir), 0.0);
fresnel = pow(fresnel, 3.0);                     float pulse = 0.85 + 0.15 *
sin(time * 12.0);                     vec3 coreC = vec3(0.53, 0.80, 1.0);
vec3 rimC = vec3(0.13, 0.27, 1.0);                     vec3 color =
mix(coreC, rimC, fresnel) * pulse;                     gl_FragColor =
vec4(color, 1.0);                 }             `,             uniforms: {
time: { value: 0 },                 spinRate: { value: 3.0 }             }
});         this.coreMesh = new THREE.Mesh(coreGeo, this.coreMaterial);
this.container.add(this.coreMesh);          // -- Layer 2: Glow Shell --
const glowMat = new THREE.ShaderMaterial({             // Use
```

```
torpedoGlow.frag from Section 4.3             vertexShader: `
varying vec2 vUv;                 void main() {                     vUv = uv;
gl_Position = projectionMatrix                        * modelViewMatrix *
vec4(position, 1.0);                }             `,
fragmentShader: `                 uniform float time;                 varying
vec2 vUv;                void main() {                     vec2 c = vUv -
0.5;                    float d = length(c) * 2.0;                     float
glow = exp(-d * d * 3.0);                     float pulse = 0.8 + 0.2 *
sin(time * 8.0);                    glow *= pulse;                     vec3
cc = vec3(0.7, 0.85, 1.0);                     vec3 ec = vec3(0.1, 0.2, 0.8);
vec3 col = mix(cc, ec, d);                     float a = glow *
smoothstep(1.0, 0.6, d);                    gl_FragColor = vec4(col * a, a);
}             `,             uniforms: { time: { value: 0 } },
transparent: true,             blending: THREE.AdditiveBlending,
depthWrite: false,             side: THREE.DoubleSide         });
this.glowSprite = new THREE.Mesh(             new THREE.PlaneGeometry(2, 2),
glowMat         );         this.glowSprite.renderOrder = 97;
this.container.add(this.glowSprite);          // -- Layer 3: Engine Trail
(ribbon) --         this.trail = new RibbonTrail(scene, camera);
this.trail.material.uniforms.noiseTexture.value = noiseTexture;          //
-- Layer 4: Particle Wake --         this.wake = new GPUParticleSystem(scene,
{             maxParticles: 200,             spawnRate: 30,
lifetime: { min: 0.3, max: 0.5 },             size: { start: 0.05, end:
0.2 },             colorStart: new THREE.Color(0xAADDFF),
colorEnd: new THREE.Color(0x1122AA),             blending:
THREE.AdditiveBlending         });     }      /**      * Launch the torpedo.
* @param {THREE.Vector3} origin - Launch position      * @param
{THREE.Vector3} direction - Initial direction (normalized)      * @param
{THREE.Object3D|null} target - Lock-on target (or null)      */
fire(origin, direction, target = null) {         this.position.copy(origin);
this.velocity.copy(direction).normalize()             .multiplyScalar(this.sp
eed);         this.target = target;         this.age = 0;         this.state
= 'straight';         this.active = true;         this.container.visible =
true;         this.container.position.copy(origin);
this.trail.reset();          // Sound hook: LAUNCH         //
AudioManager.play('torpedo_launch', origin);     }      /**      * Update
torpedo each frame.      */     update(deltaTime) {         if (!this.active)
return;         if (this.state === 'done') return;          this.age +=
deltaTime;          // Update shader time uniforms
this.coreMaterial.uniforms.time.value = this.age;
this.glowSprite.material.uniforms.time.value = this.age;          if
(this.state === 'detonating')
{             this._updateDetonation(deltaTime);
return;         }          // -- Guidance System --         if (this.state
=== 'straight' &&             this.age >= this.straightDuration) {
this.state = 'guided';         }          if (this.state === 'guided' &&
this.target) {             // Proportional navigation guidance
const targetPos = new THREE.Vector3();
this.target.getWorldPosition(targetPos);             const toTarget = new
THREE.Vector3()                 .subVectors(targetPos, this.position);
const desiredDir = toTarget.clone().normalize();             const currentDir
= this.velocity.clone().normalize();             // Steering: blend toward
target direction             const navGain = 3.0; // proportional navigation
constant             const steer = new
THREE.Vector3()                 .subVectors(desiredDir, currentDir)
.multiplyScalar(navGain);
this.velocity.add(                 steer.multiplyScalar(deltaTime *
this.speed)             );
this.velocity.normalize().multiplyScalar(this.speed);         }          //
Move torpedo         _tempVec.copy(this.velocity).multiplyScalar(deltaTime);
```

```
this.position.add(_tempVec);
this.container.position.copy(this.position);          // Orient torpedo along
velocity         const lookTarget = this.position.clone().add(this.velocity);
this.container.lookAt(lookTarget);          // Billboard the glow sprite
toward camera
this.glowSprite.lookAt(this.camera.position);          // Update trail
this.trail.pushPosition(this.position);
this.trail.rebuild(deltaTime);          // Spawn wake particles
this.wake.spawn(this.position, {             velocity:
this.velocity.clone().multiplyScalar(-0.1),             radialSpread: 0.5
});         this.wake.update(deltaTime);          // Sound hook: FLIGHT LOOP
(continuous)         // AudioManager.updateLoop('torpedo_flight',
this.position);     }      /**      * Trigger detonation at the given
position.      */     detonate(position) {         this.state = 'detonating';
this.detonationAge = 0;         this.detonationPos = position.clone();
this.container.visible = false;         this.trail.reset();          // Sound
hook: IMPACT         // AudioManager.play('torpedo_impact', position);
// Create detonation effects (see Section 4.7)
this._createDetonation(position);     }      _createDetonation(position) {
// Expanding sphere: blue-white shell         const sphereGeo = new
THREE.IcosahedronGeometry(1, 2);         const sphereMat = new
THREE.MeshBasicMaterial({             color: 0x88CCFF,
transparent: true,             blending: THREE.AdditiveBlending,
depthWrite: false,             wireframe: false,             side:
THREE.DoubleSide         });         this.detonationSphere = new
THREE.Mesh(sphereGeo, sphereMat);
this.detonationSphere.position.copy(position);
this.detonationSphere.scale.set(0.01, 0.01, 0.01);
this.detonationSphere.renderOrder = 110;
this.scene.add(this.detonationSphere);          // Shockwave ring: flat
billboard ring         const ringGeo = new THREE.RingGeometry(0.8, 1.0, 32);
const ringMat = new THREE.MeshBasicMaterial({             color: 0xAADDFF,
transparent: true,             blending: THREE.AdditiveBlending,
depthWrite: false,             side: THREE.DoubleSide         });
this.shockwaveRing = new THREE.Mesh(ringGeo, ringMat);
this.shockwaveRing.position.copy(position);
this.shockwaveRing.lookAt(this.camera.position);
this.shockwaveRing.scale.set(0.01, 0.01, 0.01);
this.shockwaveRing.renderOrder = 111;
this.scene.add(this.shockwaveRing);          // Debris particles: 30-50
orange-red particles         this.debrisParticles = new
GPUParticleSystem(this.scene, {             maxParticles: 50,
spawnRate: 0,  // burst mode             lifetime: { min: 1.0, max: 2.0 },
size: { start: 0.1, end: 0.02 },             colorStart: new
THREE.Color(0xFF6633),             colorEnd: new THREE.Color(0x331100),
gravity: -2.0,             blending: THREE.AdditiveBlending         });
// Burst spawn 40 debris particles         for (let i = 0; i < 40; i++) {
const vel = new THREE.Vector3(                 (Math.random() - 0.5) * 20,
(Math.random() - 0.5) * 20,                 (Math.random() - 0.5) * 20
);             this.debrisParticles.spawn(position,
{                 velocity: vel, radialSpread: 0             });         }
}      _updateDetonation(deltaTime) {         this.detonationAge +=
deltaTime;         const t = this.detonationAge;         const duration =
0.5;         const progress = Math.min(t / duration, 1.0);          if
(this.detonationSphere) {             // Expand from 0 to 5m radius
const radius = progress * 5.0;
this.detonationSphere.scale.set(radius, radius, radius);             // Fade:
solid -> hollow shell -> fade
this.detonationSphere.material.opacity =                 1.0 - progress *
progress;             // Switch to wireframe at 60% progress for hollow shell
```

```
look             if (progress > 0.6)
{                 this.detonationSphere.material.wireframe = true;
}         }          if (this.shockwaveRing) {             // Expand at 2x
sphere speed             const ringRadius = progress * 10.0;
this.shockwaveRing.scale.set(                 ringRadius, ringRadius,
ringRadius             );             this.shockwaveRing.material.opacity =
1.0 - progress;             this.shockwaveRing.lookAt(this.camera.position);
}          // Update debris         if (this.debrisParticles)
{             this.debrisParticles.update(deltaTime);         }          //
Camera shake parameters (pass to your camera shake system):         //
intensity: (1.0 - progress) * 0.3         // duration: 0.5s         //
falloff: exponential          if (progress >= 1.0) {             this.state =
'done';             this.active = false;
this._cleanupDetonation();         }     }      _cleanupDetonation() {
if (this.detonationSphere)
{             this.scene.remove(this.detonationSphere);
this.detonationSphere.geometry.dispose();
this.detonationSphere.material.dispose();         }         if
(this.shockwaveRing) {             this.scene.remove(this.shockwaveRing);
this.shockwaveRing.geometry.dispose();
this.shockwaveRing.material.dispose();         }         if
(this.debrisParticles)
{             this.debrisParticles.dispose();         }     }      dispose()
{         this.container.visible = false;
this.scene.remove(this.container);         this.coreMesh.geometry.dispose();
this.coreMaterial.dispose();         this.glowSprite.geometry.dispose();
this.glowSprite.material.dispose();         this.trail.dispose();
this.wake.dispose();         this._cleanupDetonation();     } }
```

4.7 Detonation Effect — Parameter Reference

Effect Component	Start State	End State	Duration
Expanding Sphere	Radius 0 m, solid blue-white, opaque	Radius 5 m, wireframe shell, transparent	0.5 s
Shockwave Ring	Radius 0 m, flat billboard ring	Radius 10 m (2× sphere speed), faded	0.5 s
Full-Screen Flash	White additive overlay, 10% opacity	0% opacity	1 frame (0.016 s)
Debris Particles	40 particles, orange-red, random velocities	Faded to dark red, gravity-affected	1.0–2.0 s
Camera Shake	Intensity 0.3, exponential falloff	Intensity 0.0	0.5 s

5. EMP Burst Rendering

The EMP Burst is an area-of-effect pulse weapon that disables electronics and shields within a radius. It is visually distinct from all other weapons: an expanding sphere with electrical arcs and a screen-space distortion post-process effect.

5.1 Geometry Approach

Element	Geometry	Purpose
Pulse Sphere	`IcosahedronGeometry` (subdivisions: 3)	Expanding wireframe shell with vertex displacement
Electrical Arcs	Animated line segments on sphere surface	Crawling lightning arcs for organic electrical feel
Distortion Pass	Full-screen post-process quad	Screen-space refraction (heat-haze) centered on EMP origin

5.2 GLSL Shaders

empBurst.vert

```
// EMP Burst Vertex Shader // Icosahedron with vertex displacement noise for organic pulsing surface  uniform float time; uniform float radius;       // current animated radius uniform float noiseScale;   // displacement intensity attribute vec3 position; attribute vec3 normal;  varying vec3 vNormal; varying vec3 vWorldPos; varying float vDisplacement;  // Simple 3D noise function (hash-based) float hash(vec3 p) {     p = fract(p * vec3(443.897, 441.423, 437.195));     p += dot(p, p.yzx + 19.19);     return fract((p.x + p.y) * p.z); }  float noise3D(vec3 p) {     vec3 i = floor(p);     vec3 f = fract(p);     f = f * f * (3.0 - 2.0 * f);     float n = mix(         mix(mix(hash(i), hash(i + vec3(1,0,0)), f.x), mix(hash(i + vec3(0,1,0)), hash(i + vec3(1,1,0)), f.x), f.y), mix(mix(hash(i + vec3(0,0,1)), hash(i + vec3(1,0,1)), f.x), mix(hash(i + vec3(0,1,1)), hash(i + vec3(1,1,1)), f.x), f.y),         f.z );     return n; }  void main() {     // Displacement along normal for organic pulsing     float noiseVal = noise3D(normal * 4.0 + time * 2.0); float displacement = noiseVal * noiseScale;     vDisplacement = displacement; vec3 displaced = position * radius + normal * displacement;      vNormal = normalize(normalMatrix * normal);     vec4 worldPos = modelMatrix * vec4(displaced, 1.0);     vWorldPos = worldPos.xyz;      gl_Position = projectionMatrix * viewMatrix * worldPos; }
```

empBurst.frag

```
// EMP Burst Fragment Shader // Electric blue wireframe pattern with crawling arc effects // Alpha fadeout as sphere expands  uniform float time; uniform
```

```
float progress;     // 0.0 = just fired, 1.0 = fully expanded uniform float
radius; uniform vec3 empColor;      // vec3(0.3, 0.6, 1.0) electric blue
uniform sampler2D noiseTexture;  varying vec3 vNormal; varying vec3
vWorldPos; varying float vDisplacement;  void main() {     // Wireframe
pattern: bright lines at triangle edges     // Use screen-space derivatives
for edge detection     vec3 dx = dFdx(vWorldPos);     vec3 dy =
dFdy(vWorldPos);     float edgeFactor = length(cross(dx, dy));     float wire
= smoothstep(0.0, 0.02, edgeFactor);      // Crawling electrical arcs using
animated noise     vec3 noiseCoord = vNormal * 3.0 + time * vec3(1.5, 0.7,
1.2);     float arcNoise = noise3D_frag(noiseCoord);     float arcs =
smoothstep(0.65, 0.7, arcNoise);      // Combine wireframe + arcs     float
intensity = max(wire * 0.4, arcs);     intensity *= (1.0 + vDisplacement *
5.0);      // Color: electric blue with white-hot arcs     vec3 color =
mix(empColor, vec3(1.0, 1.0, 1.0), arcs * 0.8);      // Alpha fadeout as
sphere expands     float alpha = (1.0 - progress * progress) * intensity;
// Fresnel edge brightening     float fresnel = 1.0 - max(dot(vNormal,
vec3(0.0, 0.0, 1.0)), 0.0);     alpha *= (0.5 + fresnel * 0.5);
gl_FragColor = vec4(color * alpha, alpha); }  // Inline noise function for
fragment shader float noise3D_frag(vec3 p) {     vec3 i = floor(p);     vec3
f = fract(p);     f = f * f * (3.0 - 2.0 * f);     float a = fract(sin(dot(i,
vec3(127.1,311.7,74.7))) * 43758.5453);     float b = fract(sin(dot(i +
vec3(1,0,0), vec3(127.1,311.7,74.7))) * 43758.5453);     float c =
fract(sin(dot(i + vec3(0,1,0), vec3(127.1,311.7,74.7))) * 43758.5453);
float d = fract(sin(dot(i + vec3(1,1,0), vec3(127.1,311.7,74.7))) *
43758.5453);     float e = fract(sin(dot(i + vec3(0,0,1),
vec3(127.1,311.7,74.7))) * 43758.5453);     float f2 = fract(sin(dot(i +
vec3(1,0,1), vec3(127.1,311.7,74.7))) * 43758.5453);     float g =
fract(sin(dot(i + vec3(0,1,1), vec3(127.1,311.7,74.7))) * 43758.5453);
float h = fract(sin(dot(i + vec3(1,1,1), vec3(127.1,311.7,74.7))) *
43758.5453);     return mix(mix(mix(a,b,f.x), mix(c,d,f.x), f.y),
mix(mix(e,f2,f.x), mix(g,h,f.x), f.y), f.z); }
```

empDistortion.frag — Post-Process Distortion Shader

```
// EMP Distortion Post-Process Fragment Shader // Screen-space UV offset
based on distance from EMP center // Creates heat-haze / refraction effect
uniform sampler2D tDiffuse;     // scene render target uniform vec2
empScreenPos;      // EMP center in screen coords (0-1) uniform float
empRadius;        // current radius in screen space uniform float
distortionStrength; uniform float time;  varying vec2 vUv;  void main() {
vec2 toCenter = vUv - empScreenPos;     float dist =
length(toCenter);      // Distortion ring: strongest at the expanding edge
float ringDist = abs(dist - empRadius);     float ring = smoothstep(0.1, 0.0,
ringDist);      // UV offset direction: radial from EMP center     vec2 dir =
normalize(toCenter);      // Apply distortion with sine wave for organic
movement     float offset = ring * distortionStrength         * sin(dist *
40.0 + time * 10.0);     vec2 distortedUV = vUv + dir * offset;      // Clamp
to valid UV range     distortedUV = clamp(distortedUV, 0.0, 1.0);      //
Slight chromatic aberration at distortion edge     vec4 colorR =
texture2D(tDiffuse, distortedUV + dir * offset * 0.3);     vec4 colorG =
texture2D(tDiffuse, distortedUV);     vec4 colorB = texture2D(tDiffuse,
distortedUV - dir * offset * 0.3);      gl_FragColor = vec4(colorR.r,
colorG.g, colorB.b, 1.0); }
```

5.3 Three.js Integration Code

EMPBurst.js

```
import * as THREE from 'three'; import { EffectComposer } from
'three/addons/postprocessing/EffectComposer.js'; import { ShaderPass } from
```

```
'three/addons/postprocessing/ShaderPass.js'; import empVertShader from
'./shaders/empBurst.vert'; import empFragShader from
'./shaders/empBurst.frag'; import empDistortionFrag from
'./shaders/empDistortion.frag';  /**  * EMPBurst - Area-of-effect
electromagnetic pulse weapon.  *  * Creates an expanding wireframe sphere
with electrical arcs  * and a screen-space distortion post-process effect.  *
* Usage:  *   const emp = new EMPBurst(scene, camera, composer);  *
emp.fire(origin, 50, 1.5);  // 50m radius, 1.5s duration  *   // In render
loop:  *   emp.update(deltaTime);  */ export class EMPBurst
{     constructor(scene, camera, composer) {         this.scene = scene;
this.camera = camera;         this.composer = composer;         this.active =
false;         this.age = 0;         this.maxRadius = 0;
this.burstDuration = 0;         this.origin = new
THREE.Vector3();          // -- Sphere Geometry --         const geo = new
THREE.IcosahedronGeometry(1, 3);         this.material = new
THREE.ShaderMaterial({             vertexShader: empVertShader,
fragmentShader: empFragShader,             uniforms: {                 time:
{ value: 0 },                 radius:     { value: 0 },
progress:   { value: 0 },                 noiseScale: { value: 0.3 },
empColor:   { value: new THREE.Vector3(0.3, 0.6, 1.0) },
noiseTexture: { value: null }             },             transparent: true,
blending: THREE.AdditiveBlending,             depthWrite: false,
wireframe: true,             side: THREE.DoubleSide         });
this.mesh = new THREE.Mesh(geo, this.material);         this.mesh.renderOrder
= 105;         this.mesh.visible = false;         scene.add(this.mesh);
// -- Distortion Post-Process Pass --         this.distortionPass = new
ShaderPass({             uniforms: {                 tDiffuse: { value:
null },                 empScreenPos: { value: new THREE.Vector2(0.5, 0.5) },
empRadius: { value: 0 },                 distortionStrength: { value: 0 },
time: { value: 0 }             },             vertexShader: `
varying vec2 vUv;                 void main() {                     vUv = uv;
gl_Position = projectionMatrix                         * modelViewMatrix *
vec4(position, 1.0);                 }             `,
fragmentShader: empDistortionFrag         });
this.distortionPass.enabled = false;         if (this.composer) {
this.composer.addPass(this.distortionPass);         }     }      /**      *
Fire the EMP burst.      * @param {THREE.Vector3} origin - Center point
* @param {number} maxRadius - Maximum radius in meters      * @param {number}
burstDuration - Expansion time in seconds      */     fire(origin, maxRadius
= 50, burstDuration = 1.5) {         this.origin.copy(origin);
this.maxRadius = maxRadius;         this.burstDuration = burstDuration;
this.age = 0;         this.active = true;
this.mesh.position.copy(origin);         this.mesh.visible = true;
this.distortionPass.enabled = true;     }      /**      * Update each frame.
*/     update(deltaTime) {         if (!this.active) return;
this.age += deltaTime;         const progress = Math.min(this.age /
this.burstDuration, 1.0);         const currentRadius = progress *
this.maxRadius;          // Update sphere
this.material.uniforms.time.value = this.age;
this.material.uniforms.radius.value = currentRadius;
this.material.uniforms.progress.value = progress;          // Update
distortion pass         // Project origin to screen space         const
screenPos = this.origin.clone()             .project(this.camera);
this.distortionPass.uniforms.empScreenPos.value.set(             (screenPos.x
+ 1) * 0.5,             (screenPos.y + 1) * 0.5         );
this.distortionPass.uniforms.empRadius.value =             progress * 0.5; //
approximate screen coverage
this.distortionPass.uniforms.distortionStrength.value =             0.02 *
(1.0 - progress);         this.distortionPass.uniforms.time.value = this.age;
// Gameplay hook: check for enemies within radius         //
```

```
GameplayManager.empDamageCheck(this.origin, currentRadius);          if
(progress >= 1.0) {             this.active = false;
this.mesh.visible = false;             this.distortionPass.enabled = false;
}     }      dispose() {         this.scene.remove(this.mesh);
this.mesh.geometry.dispose();         this.material.dispose();         if
(this.composer) {             const idx =
this.composer.passes.indexOf(this.distortionPass);             if (idx > -1)
this.composer.passes.splice(idx, 1);         }     } }
```

6. Muzzle Flash System

A shared muzzle flash system is used by all four weapon types. Each flash is a brief, bright billboard sprite at the weapon hardpoint, accompanied by a dynamic point light.

6.1 Specifications

Parameter	Laser	Scatter Shot	Proton Torpedo	EMP Burst
Flash Duration	0.05 s	0.08 s	0.12 s	0.06 s
Flash Size	0.6 m	0.8 m	1.2 m	1.0 m
Flash Color	Beam color + white	Orange-yellow (#FFAA33)	Blue-white (#88CCFF)	Electric blue (#4488FF)
Light Intensity	5.0	3.0	8.0	6.0
Light Range	10 m	8 m	15 m	12 m
Spritesheet	4×4 grid (16 frames), 0.05 s per frame, shared atlas			

6.2 Three.js Integration Code

MuzzleFlash.js

```
import * as THREE from 'three';  /**  * MuzzleFlash - Billboard spritesheet
flash at weapon hardpoint.  * Includes animated sprite + point light.  *  *
Usage:  *   const flash = new MuzzleFlash(scene, atlasTexture);  *
flash.trigger(hardpointPos, 'laser', beamColor);  *   // In render loop:  *
flash.update(deltaTime);  */ export class MuzzleFlash
{     constructor(scene, atlasTexture) {         this.scene = scene;
this.active = false;         this.age = 0;         this.duration = 0.05;
this.frameTime = 0.005;  // time per spritesheet frame
this.currentFrame = 0;         this.totalFrames = 16;         this.gridSize =
4;       // 4x4 grid          // Sprite with spritesheet
this.spriteMat = new THREE.SpriteMaterial({             map: atlasTexture,
```

```
blending: THREE.AdditiveBlending,             depthWrite: false,
transparent: true         });         this.sprite = new
THREE.Sprite(this.spriteMat);         this.sprite.renderOrder = 110;
this.sprite.visible = false;         scene.add(this.sprite);          //
Point light         this.light = new THREE.PointLight(0xFFFFFF, 0, 10);
this.light.visible = false;         scene.add(this.light);          // Set
initial UV offset for spritesheet frame 0         this._setFrame(0);     }
/**      * Trigger a muzzle flash.      * @param {THREE.Vector3} position -
Hardpoint world position      * @param {'laser'|'scatter'|'torpedo'|'emp'}
weaponType      * @param {THREE.Color} color - Weapon-specific color      */
trigger(position, weaponType, color) {         const specs =
{             laser:   { duration: 0.05, size: 0.6, intensity: 5, range:
10 },             scatter: { duration: 0.08, size: 0.8, intensity: 3, range:
8 },             torpedo: { duration: 0.12, size: 1.2, intensity: 8, range:
15 },             emp:     { duration: 0.06, size: 1.0, intensity: 6, range:
12 }         };         const spec = specs[weaponType] || specs.laser;
this.duration = spec.duration;         this.age = 0;
this.currentFrame = 0;         this.active = true;          // Position
this.sprite.position.copy(position);         this.sprite.scale.set(spec.size,
spec.size, 1);         this.sprite.visible = true;
this.spriteMat.color.copy(color);          // Light
this.light.position.copy(position);         this.light.color.copy(color);
this.light.intensity = spec.intensity;         this.light.distance =
spec.range;         this.light.visible = true;          this._setFrame(0);
}      update(deltaTime) {         if (!this.active) return;
this.age += deltaTime;          // Advance spritesheet frame         const
frameIndex = Math.floor(this.age / this.frameTime);         if (frameIndex !
== this.currentFrame             && frameIndex < this.totalFrames) {
this.currentFrame = frameIndex;             this._setFrame(frameIndex);
}          // Fade light intensity         const progress = this.age /
this.duration;         this.light.intensity *= (1.0 - progress);          //
Scale up slightly over lifetime         const scale = this.sprite.scale.x *
(1.0 + progress * 0.5);         this.sprite.scale.set(scale, scale, 1);
if (this.age >= this.duration) {             this.active = false;
this.sprite.visible = false;             this.light.visible =
false;         }     }      _setFrame(index) {         const col = index %
this.gridSize;         const row = Math.floor(index / this.gridSize);
const size = 1.0 / this.gridSize;          this.spriteMat.map.offset.set(col
* size, 1 - (row + 1) * size);         this.spriteMat.map.repeat.set(size,
size);     }      dispose() {         this.scene.remove(this.sprite);
this.scene.remove(this.light);         this.spriteMat.dispose();     } }
```

7. Impact Effect System

A shared impact system handles the visual feedback when any weapon hits a surface. It creates sparks, flashes, scorch marks, and hull damage glow effects.

7.1 Impact Components

Component	Geometry	Count	Duration	Behavior
Surface Sparks	GPU particles (Points)	8–20 per impact	0.2–0.5 s	Random hemispherical distribution along hit normal, gravity-affected
Impact Flash	Billboard sprite	1 per impact	0.1 s	Bright white-yellow flash, additive, scales up then fades
Scorch Mark	Projected decal (PlaneGeometry)	1 per impact	2.0 s (fade)	Dark circular mark on surface, oriented along hit normal
Hull Damage Glow	Point light + emissive sprite	1 per impact	2.0 s	Orange-red glow at hit location, fades out

7.2 Three.js Integration Code

ImpactEffect.js

```
import * as THREE from 'three'; import { GPUParticleSystem } from
'./GPUParticleSystem';  const _tempVec = new THREE.Vector3();  /**  *
ImpactEffect - Visual feedback when a weapon hits a surface.  *  * Usage:  *
const impact = new ImpactEffect(scene);  *   impact.spawn(hitPoint,
hitNormal, 'laser', laserColor);  *   // In render loop:  *
impact.update(deltaTime);  */ export class ImpactEffect
{     constructor(scene) {         this.scene = scene;         this.effects =
[]; // active impact instances          // Shared spark particle system
this.sparkSystem = new GPUParticleSystem(scene, {             maxParticles:
500,             spawnRate: 0,             lifetime: { min: 0.2, max: 0.5 },
size: { start: 0.04, end: 0.01 },             colorStart: new
THREE.Color(0xFFFF88),             colorEnd: new THREE.Color(0xFF4400),
gravity: -9.8,             blending: THREE.AdditiveBlending         });     }
/**      * Spawn an impact effect at the given point.      * @param
{THREE.Vector3} point - World-space hit location      * @param
{THREE.Vector3} normal - Surface normal at hit      * @param
{'laser'|'scatter'|'torpedo'|'emp'} weaponType      * @param {THREE.Color}
color      */     spawn(point, normal, weaponType, color) {         const
sparkCount = weaponType === 'torpedo' ? 20 : 12;          // Spawn sparks in
hemisphere along normal         for (let i = 0; i < sparkCount; i++) {
// Random direction in hemisphere             const theta = Math.random() *
Math.PI * 2;             const phi = Math.random() * Math.PI * 0.5;
const localDir = new THREE.Vector3(                 Math.sin(phi) *
```

```
Math.cos(theta),                 Math.sin(phi) * Math.sin(theta),
Math.cos(phi)             );              // Orient hemisphere along normal
const up = Math.abs(normal.y) < 0.9                 ? new THREE.Vector3(0, 1,
0)                 : new THREE.Vector3(1, 0, 0);             const tangent =
new THREE.Vector3()                 .crossVectors(normal, up).normalize();
const bitangent = new THREE.Vector3()                 .crossVectors(normal,
tangent);              const worldDir = new
THREE.Vector3()                 .addScaledVector(tangent, localDir.x)
.addScaledVector(bitangent,
localDir.y)                 .addScaledVector(normal, localDir.z);
const speed = 5 + Math.random() * 15;
this.sparkSystem.spawn(point, {                 velocity:
worldDir.multiplyScalar(speed),                 radialSpread:
0             });         }          // Impact flash (billboard sprite)
const flashMat = new THREE.SpriteMaterial({             color: 0xFFFFAA,
blending: THREE.AdditiveBlending,             depthWrite: false,
transparent: true         });         const flash = new
THREE.Sprite(flashMat);         flash.position.copy(point);
flash.scale.set(0.5, 0.5, 1);         flash.renderOrder = 112;
this.scene.add(flash);          // Scorch mark decal (oriented along normal)
const scorchGeo = new THREE.PlaneGeometry(0.5, 0.5);         const scorchMat
= new THREE.MeshBasicMaterial({             color: 0x111111,
transparent: true,             depthWrite: false,             side:
THREE.DoubleSide         });         const scorch = new THREE.Mesh(scorchGeo,
scorchMat);
scorch.position.copy(point)             .add(normal.clone().multiplyScalar(0.
01));         scorch.lookAt(point.clone().add(normal));
scorch.renderOrder = 50;         this.scene.add(scorch);          // Hull
damage glow         const glowLight = new
THREE.PointLight(             color, 3.0, 5         );
glowLight.position.copy(point);
this.scene.add(glowLight);          // Track for cleanup
this.effects.push({             flash, scorch, glowLight, flashMat,
scorchMat,             age: 0,             flashDuration: 0.1,
scorchDuration: 2.0,             glowDuration: 2.0         });     }
update(deltaTime) {         this.sparkSystem.update(deltaTime);          for
(let i = this.effects.length - 1; i >= 0; i--) {             const e =
this.effects[i];             e.age += deltaTime;             // Flash: scale
up then hide             if (e.age < e.flashDuration) {                 const
t = e.age / e.flashDuration;                 const s = 0.5 + t * 1.5;
e.flash.scale.set(s, s, 1);                 e.flashMat.opacity = 1.0 - t;
} else {                 e.flash.visible =
false;             }             // Scorch: fade out             if (e.age <
e.scorchDuration) {                 e.scorchMat.opacity = 1.0 - e.age /
e.scorchDuration;             }             // Glow: fade out             if
(e.age < e.glowDuration) {                 e.glowLight.intensity =
3.0 * (1.0 - e.age / e.glowDuration);             }             // Cleanup
when all components expired             if (e.age >= Math.max(
e.flashDuration, e.scorchDuration, e.glowDuration             )) {
this.scene.remove(e.flash);                 this.scene.remove(e.scorch);
this.scene.remove(e.glowLight);                 e.flashMat.dispose();
e.scorchMat.dispose();                 e.scorch.geometry.dispose();
this.effects.splice(i, 1);             }         }     }      dispose() {
this.sparkSystem.dispose();         for (const e of this.effects) {
this.scene.remove(e.flash);             this.scene.remove(e.scorch);
this.scene.remove(e.glowLight);             e.flashMat.dispose();
e.scorchMat.dispose();             e.scorch.geometry.dispose();         }
this.effects.length = 0;     } }
```

8. Particle System Architecture

A generic, reusable GPU particle system that powers all weapon effects: torpedo wakes, impact sparks, detonation debris, and muzzle flash remnants. The system uses `THREE.Points` with a ring buffer recycling strategy to minimize garbage collection.

8.1 Architecture Overview

Component	Responsibility
CPU Spawner	Pushes new particles into ring buffer, sets initial position/velocity/lifetime
GPU Vertex Shader	Updates position from velocity and age, applies size attenuation and gravity
GPU Fragment Shader	Renders soft circular gradient, interpolates color over lifetime
Ring Buffer	Fixed-size array; dead particles (age > lifetime) are recycled in-place

8.2 Complete Implementation

GPUParticleSystem.js

```
import * as THREE from 'three';  /**  * GPUParticleSystem - Reusable particle
system for all weapon VFX.  *  * Uses THREE.Points with ring buffer
recycling. Particles are  * updated primarily on the GPU via vertex shader.
*  * Options:  * @param {number} maxParticles - Ring buffer capacity  *
@param {number} spawnRate - Particles per second (0 for burst mode)  * @param
{{min:number, max:number}} lifetime - Particle lifetime range  * @param
{{start:number, end:number}} size - Point size range  * @param {THREE.Color}
colorStart - Birth color  * @param {THREE.Color} colorEnd - Death color  *
@param {number} gravity - Gravity acceleration (negative = down)  * @param
{THREE.Blending} blending - Blend mode  */ export class GPUParticleSystem {
constructor(scene, options = {}) {         this.scene = scene;
this.maxParticles = options.maxParticles || 200;         this.spawnRate =
options.spawnRate || 0;         this.lifetimeMin = options.lifetime?.min ||
0.5;         this.lifetimeMax = options.lifetime?.max || 1.0;
this.sizeStart = options.size?.start || 0.1;         this.sizeEnd =
options.size?.end || 0.02;         this.colorStart = options.colorStart ||
new THREE.Color(1, 1, 1);         this.colorEnd = options.colorEnd || new
THREE.Color(0, 0, 0);         this.gravity = options.gravity || 0;
this.blending = options.blending || THREE.AdditiveBlending;         // Ring
buffer index         this.head = 0;         this.time = 0;         //
Particle attribute arrays         const n = this.maxParticles;
```

```
this.posArray = new Float32Array(n * 3);         this.velArray = new
Float32Array(n * 3);         this.colorArray = new Float32Array(n * 4);
this.sizeArray = new Float32Array(n);         this.lifetimeArray = new
Float32Array(n); // max lifetime         this.ageArray = new Float32Array(n);
// current age          // Initialize all particles as dead (age > lifetime)
this.lifetimeArray.fill(0);         this.ageArray.fill(1);          //
Geometry         this.geometry = new THREE.BufferGeometry();
this.geometry.setAttribute('position',             new
THREE.BufferAttribute(this.posArray,
3)                 .setUsage(THREE.DynamicDrawUsage));
this.geometry.setAttribute('customColor',             new
THREE.BufferAttribute(this.colorArray,
4)                 .setUsage(THREE.DynamicDrawUsage));
this.geometry.setAttribute('size',             new
THREE.BufferAttribute(this.sizeArray,
1)                 .setUsage(THREE.DynamicDrawUsage));          // Material
this.material = new THREE.ShaderMaterial({             vertexShader: `
attribute vec4 customColor;                  attribute float size;
varying vec4 vColor;                 void main() {                     vColor
= customColor;                     vec4 mvPos = modelViewMatrix *
vec4(position, 1.0);                     gl_PointSize = size * (300.0 / -
mvPos.z);                     gl_PointSize = clamp(gl_PointSize, 1.0, 64.0);
gl_Position = projectionMatrix * mvPos;                 }             `,
fragmentShader: `                 varying vec4 vColor;                void
main() {                    // Soft circular gradient
vec2 center = gl_PointCoord - 0.5;                     float dist =
length(center) * 2.0;                     float alpha = 1.0 - smoothstep(0.0,
1.0, dist);                     alpha = alpha * alpha; // softer falloff
gl_FragColor = vec4(vColor.rgb * alpha, alpha * vColor.a);                }
`,             transparent: true,             blending: this.blending,
depthWrite: false         });          this.points = new
THREE.Points(this.geometry, this.material);         this.points.frustumCulled
= false;         this.points.renderOrder = 95;
scene.add(this.points);     }      /**      * Spawn a single particle at the
given position.      * @param {THREE.Vector3} position      * @param {Object}
opts - { velocity: THREE.Vector3, radialSpread: number }      */
spawn(position, opts = {}) {         const i = this.head;          //
Position         this.posArray[i * 3]     = position.x;
this.posArray[i * 3 + 1] = position.y;         this.posArray[i * 3 + 2] =
position.z;          // Velocity         const vel = opts.velocity || new
THREE.Vector3();         const spread = opts.radialSpread || 0;
this.velArray[i * 3]     = vel.x + (Math.random() - 0.5) * spread;
this.velArray[i * 3 + 1] = vel.y + (Math.random() - 0.5) * spread;
this.velArray[i * 3 + 2] = vel.z + (Math.random() - 0.5) *
spread;          // Lifetime         this.lifetimeArray[i] = this.lifetimeMin
+ Math.random() * (this.lifetimeMax - this.lifetimeMin);
this.ageArray[i] = 0;          // Initial color and size
this.colorArray[i * 4]     = this.colorStart.r;         this.colorArray[i * 4
+ 1] = this.colorStart.g;         this.colorArray[i * 4 + 2] =
this.colorStart.b;         this.colorArray[i * 4 + 3] = 1.0;
this.sizeArray[i] = this.sizeStart;          // Advance ring buffer head
this.head = (this.head + 1) % this.maxParticles;     }      /**      * Update
all particles. Call once per frame.      */     update(deltaTime)
{         this.time += deltaTime;          for (let i = 0; i <
this.maxParticles; i++) {             if (this.ageArray[i] >=
this.lifetimeArray[i]) continue;              // Advance age
this.ageArray[i] += deltaTime;             const t = this.ageArray[i] /
this.lifetimeArray[i];              if (t >= 1.0) {                 // Dead:
hide by zeroing alpha and size                 this.colorArray[i * 4 + 3] =
0;                 this.sizeArray[i] = 0;                 continue;
```

```
}              // Update position from velocity             this.posArray[i *
3]     += this.velArray[i * 3]     * deltaTime;             this.posArray[i *
3 + 1] += this.velArray[i * 3 + 1] * deltaTime;             this.posArray[i *
3 + 2] += this.velArray[i * 3 + 2] * deltaTime;              // Apply gravity
to velocity Z component             this.velArray[i * 3 + 2] += this.gravity
* deltaTime;              // Interpolate color over lifetime
this.colorArray[i * 4]     =
THREE.MathUtils.lerp(this.colorStart.r, this.colorEnd.r, t);
this.colorArray[i * 4 + 1] =
THREE.MathUtils.lerp(this.colorStart.g, this.colorEnd.g, t);
this.colorArray[i * 4 + 2] =
THREE.MathUtils.lerp(this.colorStart.b, this.colorEnd.b, t);
this.colorArray[i * 4 + 3] = 1.0 - t * t; // fade alpha              //
Interpolate size             this.sizeArray[i] =
THREE.MathUtils.lerp(this.sizeStart, this.sizeEnd, t);         }          //
Flag buffers for GPU upload
this.geometry.attributes.position.needsUpdate = true;
this.geometry.attributes.customColor.needsUpdate = true;
this.geometry.attributes.size.needsUpdate = true;     }      dispose() {
this.scene.remove(this.points);         this.geometry.dispose();
this.material.dispose();     } }
```

9. Performance Optimization Guide

9.1 Object Pooling

Every weapon VFX class should use an object pool to avoid allocation during gameplay. Create all instances at initialization, deactivate them, and recycle them on demand.

```
// Generic object pool pattern export class VFXPool
{     constructor(factory, initialSize) {         this.pool = [];
this.active = [];         for (let i = 0; i < initialSize; i++) {
const obj = factory();             obj.active = false;
this.pool.push(obj);         }     }      acquire() {         let obj =
this.pool.find(o => !o.active);         if (!obj)
{             console.warn('VFXPool exhausted, oldest recycled');
obj = this.active.shift();         }         obj.active = true;
this.active.push(obj);         return obj;     }      release(obj)
{         obj.active = false;         const idx = this.active.indexOf(obj);
if (idx > -1) this.active.splice(idx, 1);     }      update(dt) {         for
(let i = this.active.length - 1; i >= 0; i--)
{             this.active[i].update(dt);             if (!
this.active[i].active) {                 this.release(this.active[i]);
}         }     } }
```

9.2 Recommended Pool Sizes

VFX Type	Pool Size	Rationale
LaserBeam	20	Multiple ships firing simultaneously
ScatterShot	10	Each shot is 8–12 pellets in one InstancedMesh
ProtonTorpedo	8	Expensive multi-layer; limit concurrent count
EMPBurst	3	Rare weapon; post-process cost is high
MuzzleFlash	30	One per weapon firing; frequent
ImpactEffect	40	Every hit generates one; high frequency

9.3 InstancedMesh vs. Points vs. Individual Meshes

Technique	Best For	Draw Calls	GPU Cost
`InstancedMesh`	Scatter pellets, debris with complex shapes	1 per batch	Low–Medium
`THREE.Points`	Particle systems (sparks, wake, small particles)	1 per system	Low
Individual `Mesh`	Unique geometry per effect (torpedo core, EMP sphere)	1 per mesh	Medium
`THREE.Sprite`	Single billboard (muzzle flash, impact flash)	1 per sprite	Low (but no batching)

💡 **Tip — Draw Call Budget**

Target a maximum of **10 draw calls** for all active weapon VFX combined. This is achievable by using one `InstancedMesh` for all active scatter pellets, one `Points` geometry per particle system type, and individual meshes only for unique-geometry effects (torpedo core, EMP sphere).

9.4 LOD Strategy

Distance from Camera	Particle Count	Trails	Glow Sprites
< 100 m	100% (full budget)	Full 30-segment ribbon	Full resolution
100–300 m	50%	15-segment ribbon	Half resolution
300–500 m	25%	Disabled (use sprite instead)	Quarter resolution
> 500 m	Billboard sprite only	Disabled	Single point

9.5 Additional Optimizations

- **Shader Pre-Warming:** Compile all weapon shaders during the loading screen by rendering a single invisible frame with each material. This avoids shader compilation stutter during gameplay.
- **Texture Atlas:** Pack all VFX sprites (muzzle flash frames, impact sprites, glow textures, noise textures) into a single 2048×2048 atlas. This allows all VFX to share one texture binding.
- **Ring Buffer Modulo:** Always use `(index + 1) % capacity` for ring buffer advancement. A common bug is using `index++` without wrapping, which causes array out-of-bounds or geometry explosion.
- **GPU Timer Queries:** Use `EXT_disjoint_timer_query_webgl2` to measure GPU time per VFX pass. Target < 2ms for all VFX combined.

- **Avoid `new` in hot paths:** Pre-allocate all `THREE.Vector3`, `THREE.Matrix4`, and `THREE.Color` objects used in update loops. Create them once in the constructor and reuse with `.copy()` or `.set()`.

10. Integration Checklist

Follow these steps to integrate all weapon VFX into your Starfighter project:

```
const vfxManager = new WeaponVFXManager(scene, camera, renderer);
vfxManager.init(); // pre-warms shaders, creates pools
vfxManager.registerWeapon('laser', LaserBeam, 20);
vfxManager.registerWeapon('scatter', ScatterShot, 10);
vfxManager.registerWeapon('torpedo', ProtonTorpedo, 8);
vfxManager.registerWeapon('emp', EMPBurst, 3);
// In your weapon controller: weaponController.on('fire', (type, origin,
direction, target) => {     vfxManager.fire(type, origin, direction, target);
});
// In your physics/raycast system: hitDetection.on('hit', (point, normal,
weaponType, color) => {     vfxManager.impact(point, normal, weaponType,
color); });
function animate() {     const dt = clock.getDelta();
gameplayManager.update(dt);     vfxManager.update(dt);     // <-- after
gameplay, before render     renderer.render(scene, camera);     // Or:
composer.render() if using post-processing
requestAnimationFrame(animate); }
function cleanup() {     vfxManager.dispose(); // releases all pools,
geometry, materials }
```

> 📝 **Note — EffectComposer Requirement**
>
> If using the EMP Burst weapon, you **must** set up `EffectComposer` from `three/addons/postprocessing/` and pass it to the `EMPBurst` constructor. The distortion effect will not work with a basic `renderer.render()` call. The `EffectComposer` should also include a `RenderPass` as the first pass, followed by any bloom pass, then the EMP distortion pass, and finally an output pass.

11. Troubleshooting — Common AI Coding Mistakes

This section specifically addresses errors that VS Code AI assistants (Copilot, Codeium, Cursor, etc.) commonly produce when generating weapon VFX code for this game. If your generated code has one of these problems, refer to the corresponding section for the correct approach.

Symptom	Root Cause	Correct Fix	Reference
“Laser renders as thin line”	AI used `THREE.Line` or `LineBasicMaterial` instead of a billboard quad	Use a 4-vertex billboard quad with `ShaderMaterial`, compute cross product for camera-facing orientation	Section 2.1–2.4
“Torpedo trail is a series of dots”	AI used `THREE.Points` or individual sprites for the trail instead of a triangle strip ribbon	Use a `BufferGeometry` triangle strip with position ring buffer, rebuild geometry each frame	Section 4.4
“VFX not visible”	Missing `depthWrite: false`, wrong blending mode, or `renderOrder` too low	Set `depthWrite: false`, `blending: THREE.AdditiveBlending`, `transparent: true`, and `renderOrder > 90`	Section 1.4
“Performance drops with many projectiles”	Creating new `Mesh`/`Material` objects per projectile instead of using pools	Use object pooling (Section 9.1) and `InstancedMesh` for batch rendering	Section 9.1–9.2
“Glow looks blocky”	Non-power-of-2 textures or missing texture filtering	Use power-of-2 dimensions (256×256, 512×512), set `texture.minFilter = THREE.LinearMipma`	Section 9.5

Symptom	Root Cause	Correct Fix	Reference
		`pLinearFilter`	
“Particles don’t face camera”	Using `MeshBasicMaterial` on a plane without billboard logic	Either use `THREE.Sprite`/`SpriteMaterial`, or compute billboard orientation in vertex shader	Section 8.2
“EMP distortion not working”	Rendering directly without `EffectComposer`	Set up `EffectComposer` with `RenderPass` + `ShaderPass` for the distortion shader	Section 5.3
“Colors look wrong”	Mismatched color space settings	Set `renderer.outputColorSpace = THREE.SRGBColorSpace` and `renderer.toneMapping = THREE.ACESFilmicToneMapping`	Section 1.4
“Trail geometry explodes”	Ring buffer index not wrapping correctly	Always compute index as `((head - offset) + capacity * 2) % capacity` to handle negative modulo	Section 4.4
“Additive blending too bright / washed out”	Emissive intensities too high; no tone mapping	Reduce color intensities (multiply by 0.3–0.5), enable `ACESFilmicToneMapping`, consider HDR render targets	Section 9.5

⚠️ **Warning — The “Cylinder Beam” Antipattern**

AI assistants frequently generate a `CylinderGeometry` for laser beams. This is **wrong** because: (1) it uses far more vertices than needed, (2) it does not billboard toward the camera (looks flat from certain angles), (3) it cannot produce the soft-edge glow effect that a UV-mapped billboard quad achieves.

Always use the billboard quad approach from Section 2.

12. Glossary

Term	Definition
Additive Blending	A blend mode where the fragment color is added to the existing framebuffer color, making bright areas brighter. Used for glowing effects like lasers and explosions. In Three.js: `THREE.AdditiveBlending`.
Billboard	A flat polygon (usually a quad) that always rotates to face the camera, regardless of its position in 3D space. Used for sprites, glow effects, and beam weapons.
Buffer Geometry	Three.js geometry representation that stores vertex data in typed arrays (`Float32Array`) for direct GPU upload. More performant than the deprecated `Geometry` class.
Cross Product	A vector operation that produces a vector perpendicular to two input vectors. Used to compute billboard orientation axes. In GLSL: `cross(a, b)`.
depthWrite	Controls whether a material writes to the depth buffer. Set to `false` for transparent/additive VFX to prevent them from occluding other transparent objects.
EffectComposer	A Three.js post-processing framework that renders the scene to an off-screen buffer, applies shader passes (blur, distortion, bloom), then outputs to screen.
Fresnel Effect	An optical phenomenon where surfaces appear brighter at glancing angles. In shaders, approximated as `pow(1.0 - dot(normal, viewDir), exponent)`. Used for torpedo rim glow.

Term	Definition
GPU Particles	A particle system where position updates and rendering occur primarily on the GPU via vertex/fragment shaders, minimizing CPU overhead.
Hardpoint	A defined attachment point on the ship model where weapons are mounted. Weapon VFX originate from hardpoint world-space positions.
Hitscan	A weapon that has no travel time; it instantly checks for a hit along a ray from the origin to max range. The laser beam is a hitscan weapon.
InstancedMesh	A Three.js optimization that renders multiple copies of the same geometry with different transforms in a single draw call. Each instance has its own 4×4 matrix.
Manifold Surface	A surface where every point has a well-defined neighborhood. In the context of VFX, storing state on manifold geometry (like a triangle strip ribbon) rather than as discrete data points.
Object Pooling	A pattern where objects are pre-allocated and recycled rather than created and garbage-collected on demand. Essential for 60fps performance in JavaScript.
Premultiplied Alpha	An alpha blending approach where RGB values are pre-multiplied by the alpha channel before output. Produces correct results with additive blending.
Proportional Navigation	A guidance law where the missile turns at a rate proportional to the rotation rate of the line-of-sight to the target. Used for torpedo homing behavior.
Quad Strip	Two triangles forming a rectangle. The fundamental geometry for billboard beams and trail segments. Uses 4 vertices and 6 indices.
renderOrder	A Three.js property that controls the rendering sequence of objects. Higher values render later (on top). VFX typically use values 90–120.

Term	Definition
Ribbon Geometry	A triangle strip that forms a continuous flat surface along a path. Each segment has two vertices (left/right edges). Used for torpedo engine trails.
Ring Buffer	A fixed-size circular array where the write pointer wraps around to the beginning when it reaches the end. Used for particle recycling and trail position history.
Shader Material	A Three.js material that accepts custom GLSL vertex and fragment shader code, providing full control over rendering. Used for all weapon VFX.
smoothstep	A GLSL function that performs Hermite interpolation between two values, producing smooth transitions. Used for soft edges and gradients in VFX shaders.
Tone Mapping	The process of mapping HDR (high dynamic range) color values to the displayable LDR range. `ACESFilmicToneMapping` is recommended for VFX-heavy scenes.
Triangle Strip	A GPU primitive where each new vertex forms a triangle with the two preceding vertices. Memory-efficient for continuous surfaces like ribbons.
UV Coordinates	2D texture coordinates (U horizontal, V vertical) mapped to vertices. Range 0.0–1.0. Used for texture sampling and parametric effects (fadeout, energy scrolling).

Powered by ButterflyFX™ | kensgames.com | © 2026 Ken’s Games. All rights reserved.

This document is the single source of truth for Starfighter weapon visual effects. When in doubt, follow this guide.